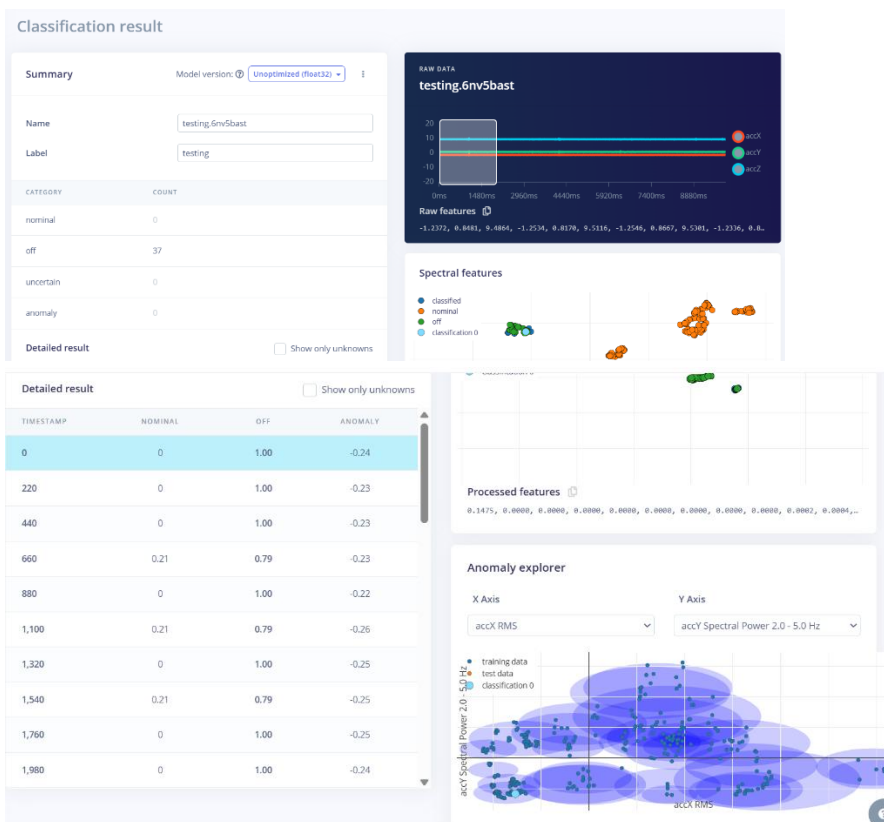


Part 1 Content: <https://studio.edgeimpulse.com/public/989432/live>

1. Raw inputs: State the raw inputs used in the Edge Impulse project.
The raw inputs used in the Edge Impulse project were time-series accelerometer measurements collected from the onboard accelerometer sensor.
2. NN classifier features: State the type of features passed into the NN classifier, the number of input features, and at least five actual feature names used to train the classifier.
The neural network classifier used Spectral Analysis features generated from the accelerometer vibration signals. The neural network classifier used 33 input features. Examples of actual features used in training include: accX RMS , accX Peak 1 Height, accY Spectral Power 2.0 - 5.0, accZ Spectral Power 2.0 - 5.0, accY Peak 1 Freq , and accZ RMS. These features were extracted from the spectral analysis DSP block.
3. NN classifier outputs: State the outputs of the NN classifier.
The outputs of the neural network classifier were nominal and off.
4. Anomaly detector features: State the type of features passed into the anomaly detector, the number of features used, and the feature names used to train the anomaly detector.
The anomaly detector used spectral analysis features extracted from the accelerometer vibration data collected by the Arduino Nano 33 BLE Sense. Four features were selected for training: accX RMS, accX Peak 1 Height, accY Spectral Power 2.0–5.0 Hz, and accZ Spectral Power 2.0–5.0 Hz. The anomaly detector was trained using 32 clusters to model normal fan behavior and identify unusual vibration patterns.
5. Deployment evidence: Include a screenshot showing the Edge Impulse model running on the Arduino Nano 33 BLE Sense and producing inference results.



```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 615 us, anomaly 519 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.945312
  off: 0.054687
Anomaly prediction: 400.838287
```

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 566 us, anomaly 562 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.968750
  off: 0.031250
Anomaly prediction: 12.188734
```

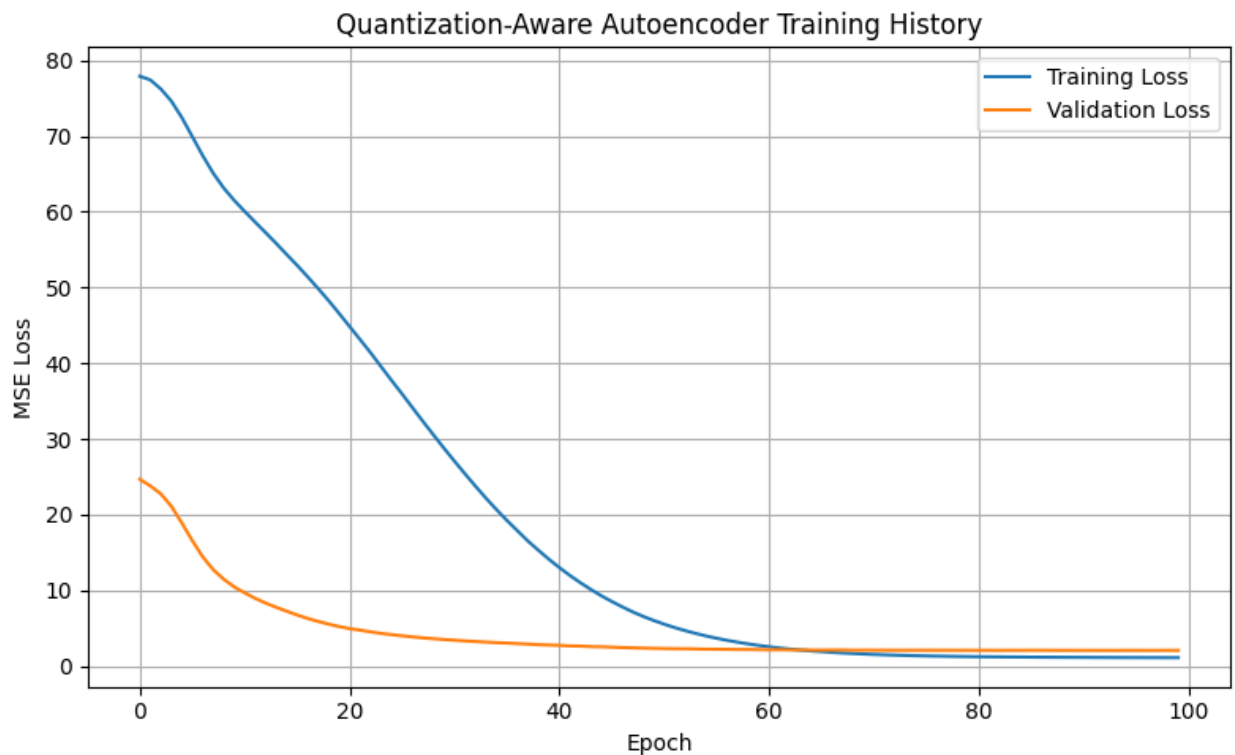
```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 593 us, anomaly 551 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.027344
  off: 0.972656
Anomaly prediction: 9.507980
```

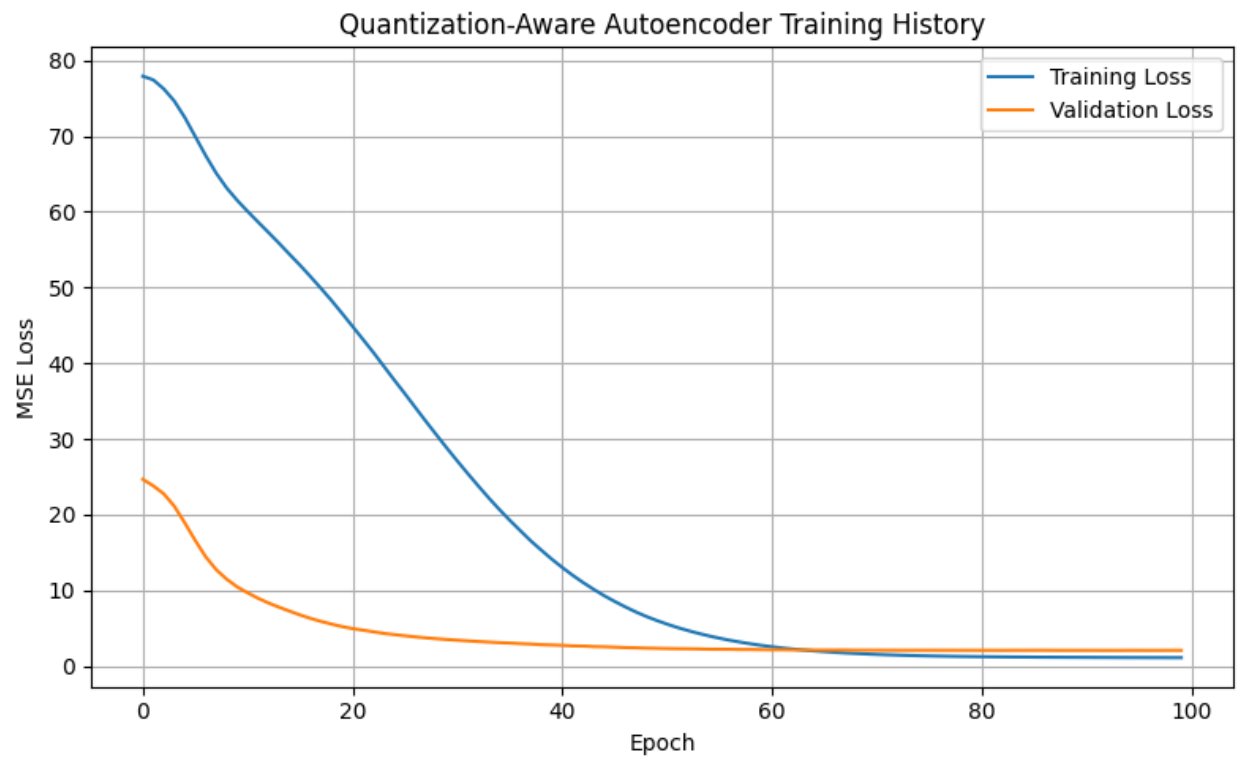
6. Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier outputs should always be trusted and under what conditions it may or may not be reliable.

The deployment results were generally accurate during testing on the Arduino Nano 33 BLE Sense. When the board was held still, the model correctly classified the state as “off,” and when the device was shaken or moved irregularly, the anomaly detector identified the motion as anomalous behavior. When we simulated fan vibrations by steadily vibrating the device, the classifier correctly detected the “nominal” operating condition. However, the classifier output should not always be fully trusted, since unexpected movements, sensor noise, or operating conditions different from the training data may reduce prediction reliability.

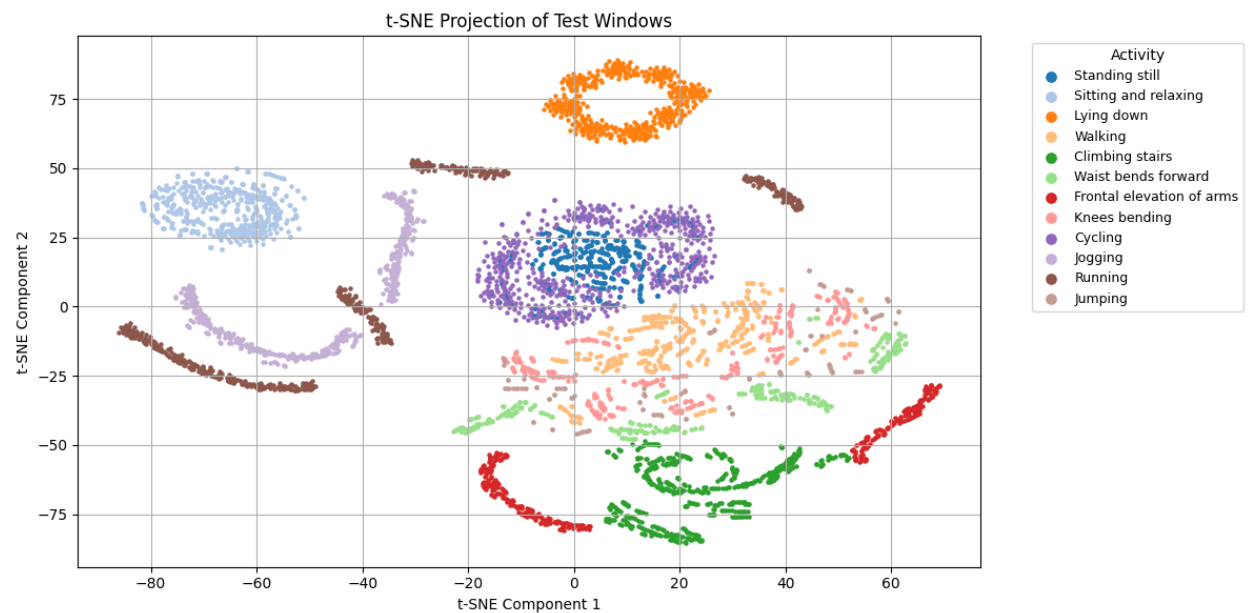
Part 2

Training and validation loss curves

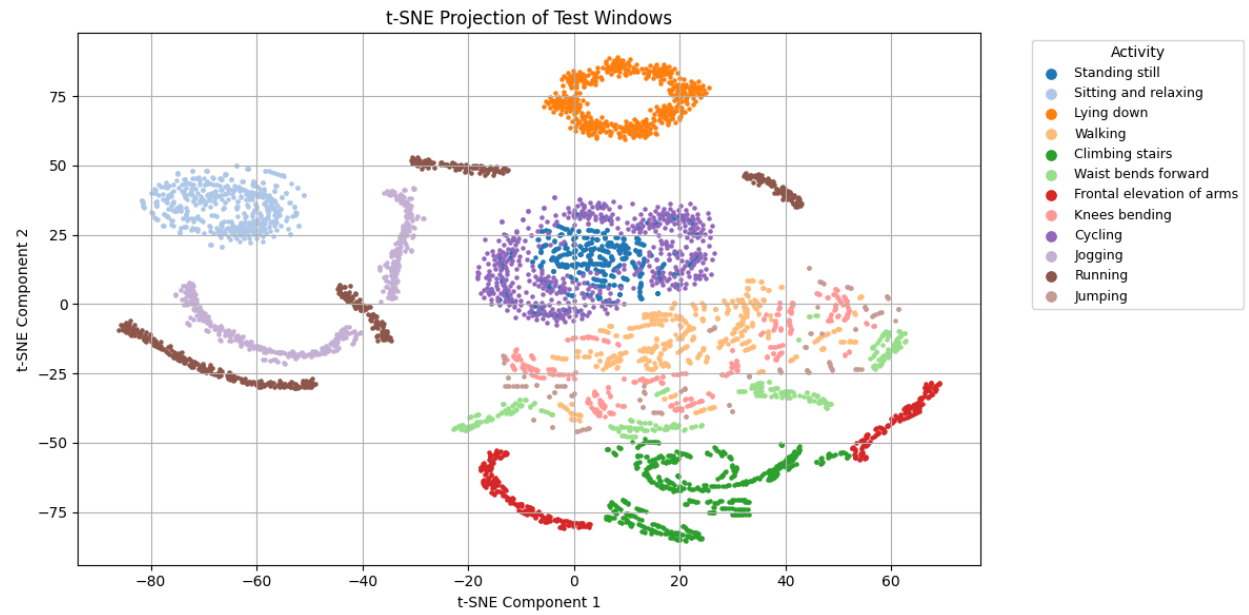




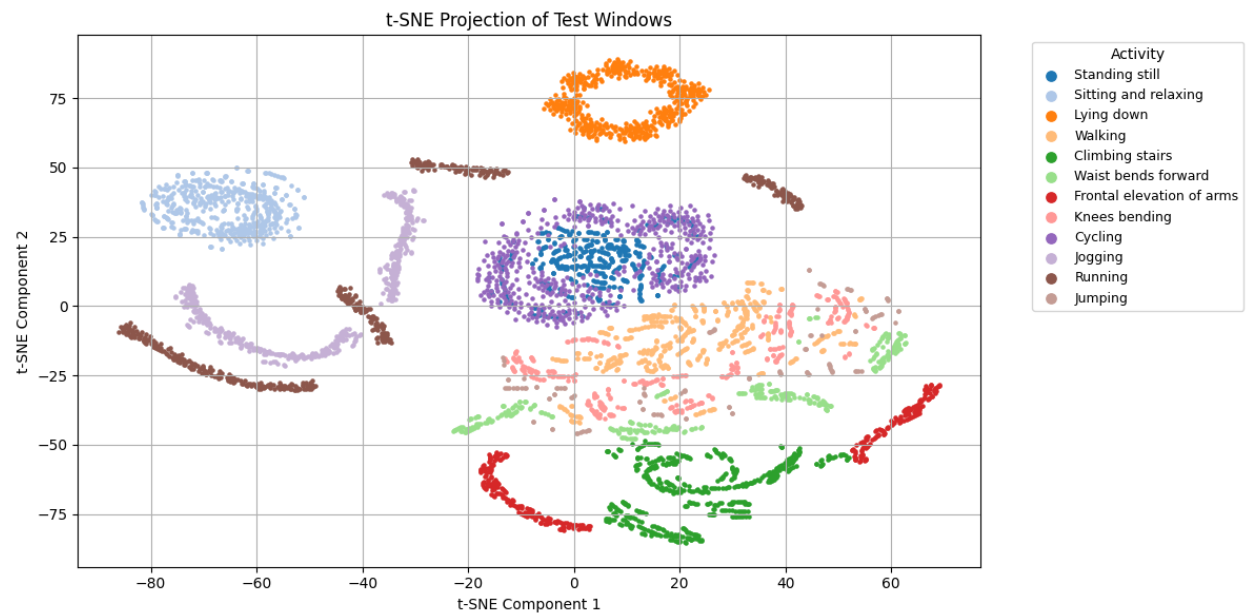
- PCA visualization



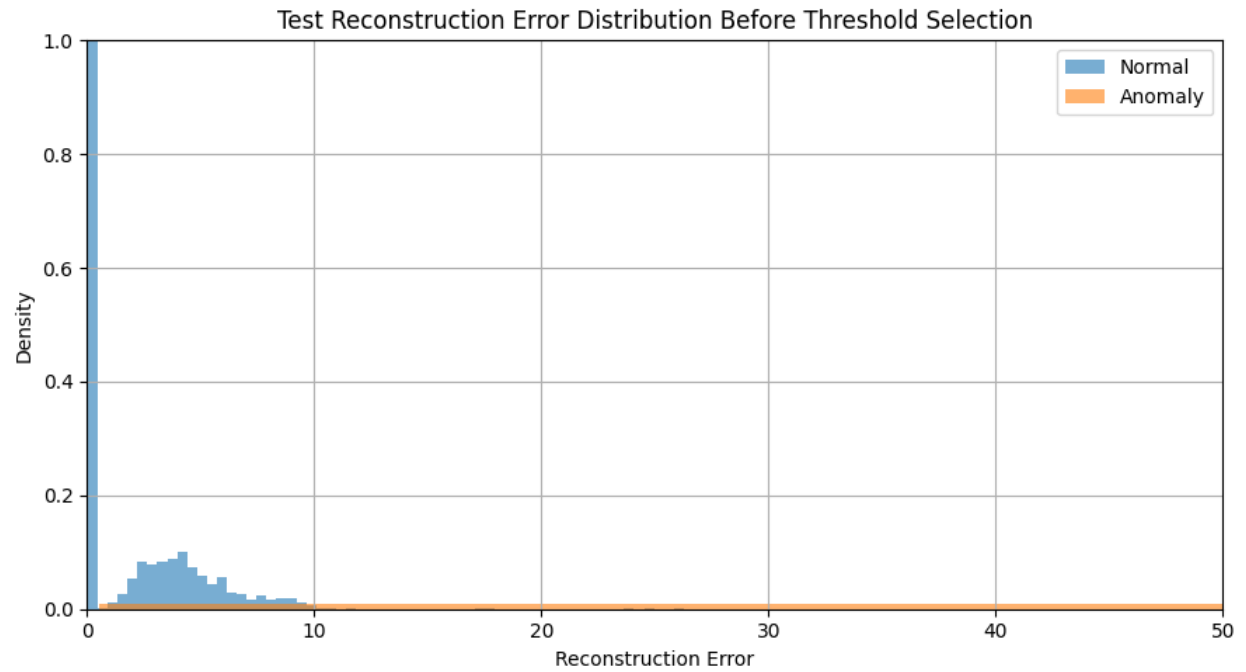
- t-SNE visualization



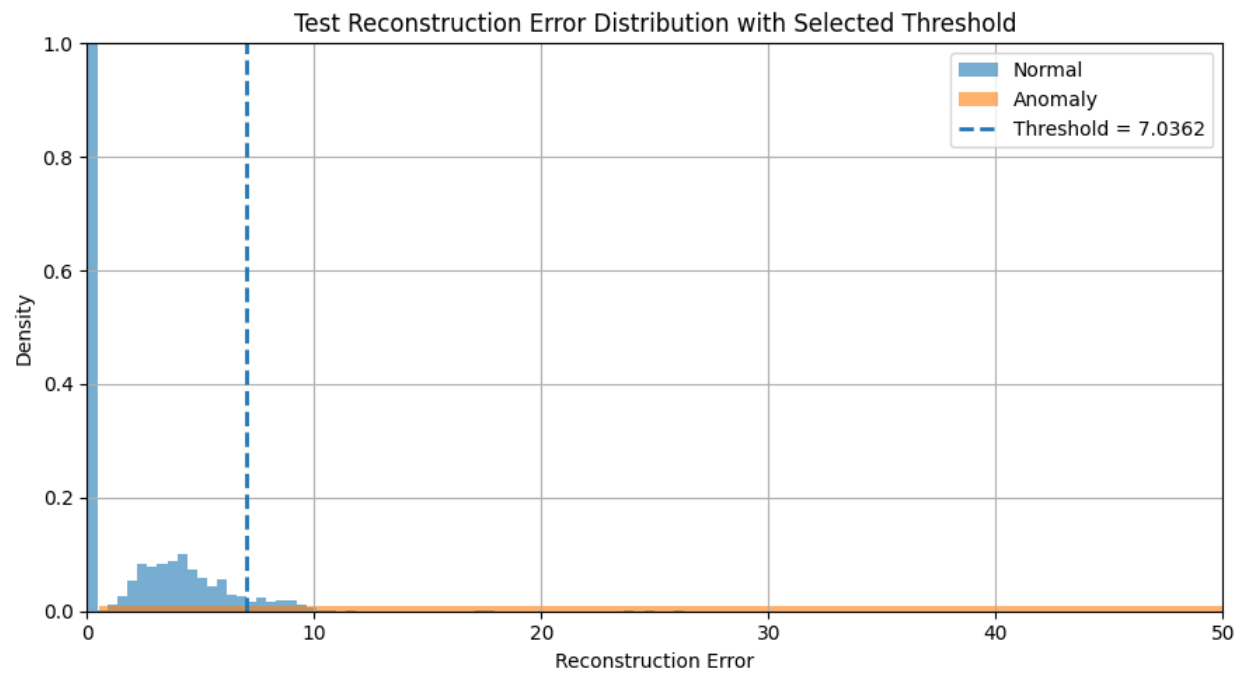
- UMAP visualization



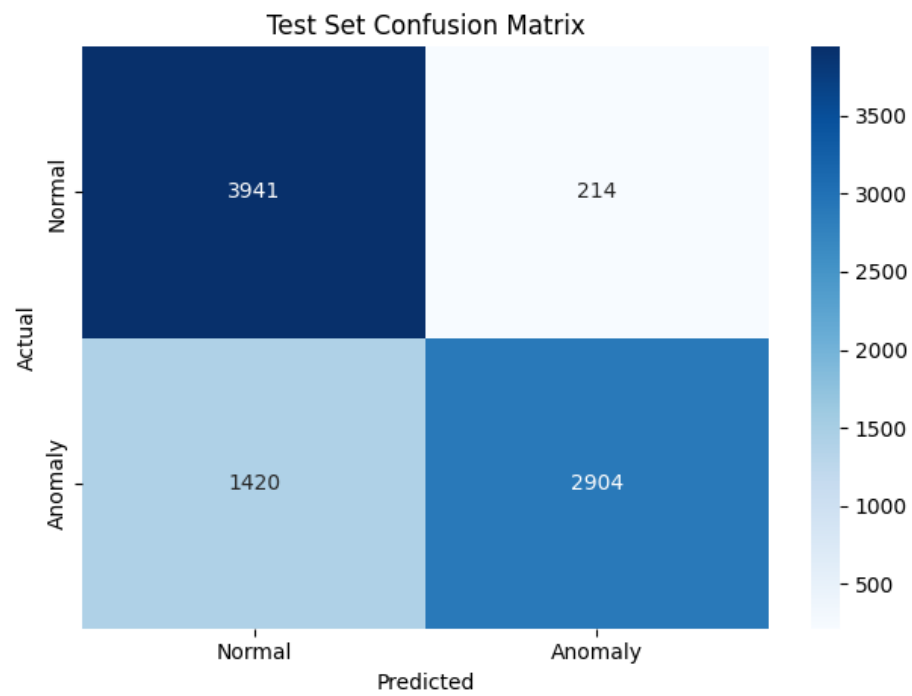
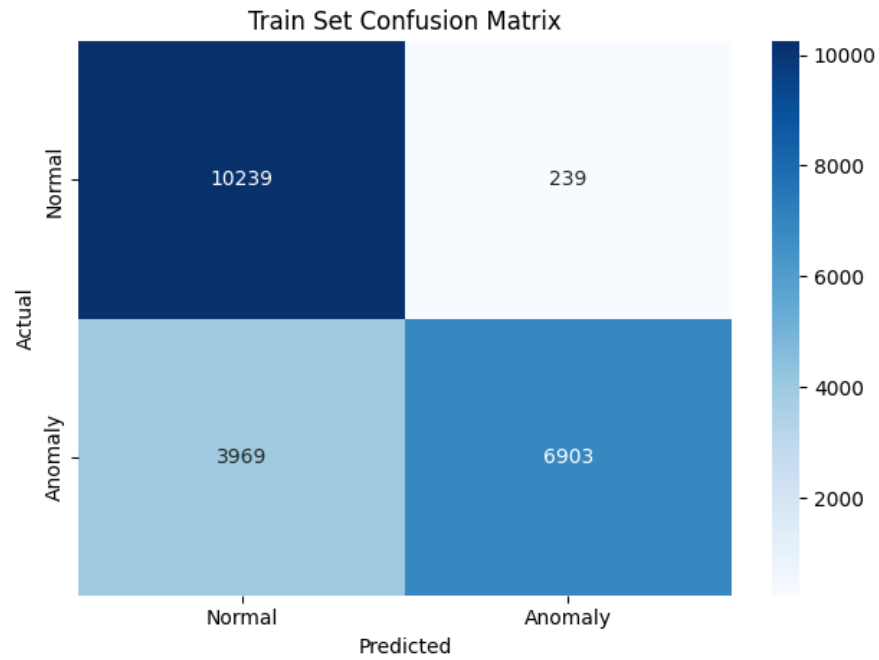
- Reconstruction error distribution



- Selected anomaly threshold



- Confusion matrix



- Classification report or evaluation metrics

Train set evaluation

	precision	recall	f1-score	support
Normal (0)	0.72	0.98	0.83	10478
Anomaly (1)	0.97	0.63	0.77	10872
accuracy			0.80	21350
macro avg	0.84	0.81	0.80	21350
weighted avg	0.85	0.80	0.80	21350

Test set evaluation

	precision	recall	f1-score	support
Normal (0)	0.74	0.95	0.83	4155
Anomaly (1)	0.93	0.67	0.78	4324
accuracy			0.81	8479
macro avg	0.83	0.81	0.80	8479
weighted avg	0.84	0.81	0.80	8479

- Arduino serial monitor output showing reconstruction error and normal/anomaly detection results

Message (Enter to send message to 'Arduin

```
Result: Normal activity
Reconstruction error: 0.102427
Result: Normal activity
Reconstruction error: 0.126232
Result: Normal activity
Reconstruction error: 0.116161
Result: Normal activity
Reconstruction error: 0.112946
Result: Normal activity
Reconstruction error: 0.136807
Result: Normal activity
```



```
Result: Anomaly detected
Reconstruction error: 2.415529
Result: Anomaly detected
Reconstruction error: 2.724370
Result: Anomaly detected
Reconstruction error: 2.741652
Result: Anomaly detected
Reconstruction error: 2.808478
Result: Anomaly detected
Reconstruction error: 3.310999
Result: Anomaly detected
```

Discussion Questions

1. What threshold did you choose for anomaly detection, and why?

A reconstruction-error threshold of approximately 7.04 was selected. This value was computed as the mean normal reconstruction error plus three standard deviations, which provided good separation between normal and anomalous activity distributions.

2. How does reconstruction error help distinguish normal and anomalous activity?

Reconstruction error measures how well the autoencoder can recreate an input signal. Since the model was trained only on normal activities, normal motion patterns produce low reconstruction error, while unfamiliar or anomalous activities produce much larger errors.

3. What information from the Python notebook must be preserved when deploying the model to Arduino?

The Arduino deployment must preserve the trained TFLite model, quantization parameters (input/output scale and zero point), reconstruction-error threshold, input window size, and preprocessing assumptions used in the Python notebook.

4. Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

The Arduino sketch must use the same preprocessing assumptions so the live sensor data matches the format and scale used during training. If preprocessing differs, the model may produce inaccurate reconstruction errors and unreliable anomaly detection.

5. What are the main limitations of this anomaly detection method when deployed on a microcontroller?

The main limitations include limited memory and compute resources, sensitivity to sensor noise and placement, reduced generalization to unseen activities, and possible differences between real-time sensor data and the original training dataset.